

ENGRAM README

README

Quick-start reference for the ENGRAM / PYRI AI persona framework.

Overview

ENGRAM is a full-stack dashboard for PYRI, an AI companion built on a fictional cognitive architecture: layered memory, a belief registry, personas, a symbolic Hiero-Code language, an emotive micro-expression layer, and a multi-mode streaming chat. Personas can also act autonomously, initiating their own messages over time.

Features

- **Layered memory** semantic, working, episodic, preference, reflective, and procedural layers.
- **Belief registry** revisable, evidence-scored hypotheses with counterarguments and revisions.
- **Personas** cognitive forms that reshape retrieval, reasoning, and tone.
- **Hiero-Code** a symbolic glyph language with a compound-expression builder.
- **Emotive Expression Layer** ASCII micro-expressions mapped to valence and arousal, woven into chat.
- **Autonomy engine** personas accrue drive pressure over time and self-initiate transmissions, with strict cost caps.
- **Multi-mode chat** seven modes, streaming responses, custom persona upload, and persisted conversations.
- **Provider-agnostic** swap between a hosted model and a fully local/offline model via configuration.
- **Responsive UI** usable on desktop and Android phones without losing the cyberpunk aesthetic.

Tech Stack

Workspace	pnpm workspaces, Node.js 24, TypeScript 5.9
API	Express 5
Database	PostgreSQL + Drizzle ORM
Validation	Zod (zod/v4) + drizzle-zod
API codegen	Orval (from the OpenAPI spec)
Frontend	React + Vite, Tailwind v4, shadcn/ui, wouter, TanStack Query

Prerequisites

- Node.js 24 and pnpm.
- A local PostgreSQL database (connection string in DATABASE_URL).
- For fully offline use: a local OpenAI-compatible model runtime (Ollama, LM Studio, llama.cpp, or vLLM).
- Optional: ffmpeg + ffprobe on PATH for the video media modality (text, image, and audio work without them).

Quick Start

Option A — Run fully local on one port (recommended)

One command builds the dashboard and the API and runs them together on a single port (default `http://localhost:5000`), with the API serving the built dashboard. The first run also installs dependencies, creates the database schema, and seeds reference data.

```
# Linux / macOS
./scripts/local/start.sh

# Windows (PowerShell)
.\scripts\local\start.ps1
```

Windows users can also double-click `scripts\local\start.bat`. Run the one-time setup on its own with the matching `setup.sh` / `setup.ps1`. Configuration lives in `.env` (copied from `.env.example` on first run) — point `DATABASE_URL` at your local Postgres and `LLM_BASE_URL` at your local model.

Option B — Development (separate servers, live reload)

1. Install dependencies from the repo root:

```
pnpm install
```

2. Provide configuration (at minimum a database URL):

```
export DATABASE_URL=postgres://user:pass@host:5432/engram
# optional: target a local model
export LLM_BASE_URL=http://localhost:11434/v1
export LLM_MODEL=llama3.1
export LLM_API_KEY=local
```

3. Create the schema and seed reference data:

```
pnpm --filter @workspace/db run push
pnpm --filter @workspace/scripts run seed:expressions
pnpm --filter @workspace/scripts run seed:engrams
pnpm --filter @workspace/scripts run seed:hub
```

4. Run the API and the dashboard:

```
pnpm --filter @workspace/api-server run dev
pnpm --filter @workspace/engram run dev
```

Configuration

`DATABASE_URL`

Required. PostgreSQL connection string.

`PORT`

Port the server listens on (default 5000 in local mode).

LLM_BASE_URL	OpenAI-compatible endpoint; set it to a local server for offline use.
LLM_API_KEY	Endpoint key; local servers accept any non-empty value.
LLM_MODEL	Model name. Set it to an installed local model when running offline.
LLM_VISION_MODEL / LLM_TRANSCRIBE_MODEL	Optional models for image/video vision and audio/video transcription.
WEB_DIST	Optional. Absolute path to the built dashboard (artifacts/engram/dist/public). When set, the API also serves the dashboard for single-port local runs; set automatically by the local start scripts.
AI_INTEGRATIONS_OPENAI_*	Optional cloud fallback used when LLM_* is unset.

Note

Leave the LLM_* variables unset to use the hosted model, or set them to point at any local OpenAI-compatible server for fully offline operation. No code changes are required either way.

Project Structure

```
artifacts/engram      # React + Vite dashboard
artifacts/api-server  # Express API + autonomy engine
lib/api-spec          # OpenAPI source of truth
lib/api-zod           # generated Zod schemas
lib/api-client-react  # generated React Query hooks
lib/db                # Drizzle schema + client
scripts               # seed + utility scripts
```

Common Commands

Run fully local (one port)	./scripts/local/start.sh (Windows: .\scripts\local\start.ps1)
Typecheck	pnpm run typecheck
Build	pnpm run build
Regenerate API	pnpm --filter @workspace/api-spec run codegen
Push schema	pnpm --filter @workspace/db run push
Seed expressions	pnpm --filter @workspace/scripts run seed:expressions
Seed engrams	pnpm --filter @workspace/scripts run seed:engrams
Seed hub / spaces	pnpm --filter @workspace/scripts run seed:hub

Notes & Gotchas

- A fresh schema push does not seed data — run the seed scripts (they are idempotent). The local start script seeds on first run.

- Re-run codegen after editing the OpenAPI spec; push after schema changes.
- Fonts are self-hosted via @fontsource — do not re-add CDN font references.
- The persona expression intimacy axis is sanitized before reaching the model; behavior stays platonic.
- Do not run pnpm dev at the workspace root; run individual packages, the local start script, or platform workflows.
- Video media perception needs ffmpeg + ffprobe on PATH; without them, video jobs fail (text/image/audio still work).
- Image/video understanding and audio/video transcription need a model that supports vision and speech-to-text; a text-only local model still handles chat and transmissions.

License

Add your chosen license here. Until a license is specified, treat this code as proprietary to the project owner.

